

Deliverable 2: Scientific Calculator

Support for F6: $B(x, y)$ (Beta Function)

Divy Soni (40297236)

SOEN 6011

August 2026

Agenda

- ① From D1 to D2: What Changed
- ② Problem 5: From-Scratch Algorithm & Subordinate Functions
- ③ Problem 5: Tkinter GUI and Exception Handling
- ④ Problem 5: Verification Cases
- ⑤ Problem 6: Version Control
- ⑥ Problem 7: Updated Requirements
- ⑦ GAI/CASTROFF Evidence
- ⑧ Attribution

From D1 to D2: What Changed

D1 (Problem 4) computed $B(x, y)$ via Simpson's Rule using Python's built-in `**` operator and console I/O.

D2 (Problem 5) required rebuilding this from scratch, with three concrete changes:

Aspect	D1	D2
Interface	Text (CLI)	Graphical (Tkinter)
Exponentiation	Built-in <code>**</code>	<code>real_power()</code> from scratch
Error handling	Generic error print	Custom exception hierarchy

Why this matters: $B(x, y)$'s integrand $t^{x-1}(1-t)^{y-1}$ needs real-valued exponentiation, not just integer powers, so removing `**` meant building `exp` and `ln` from scratch first, then composing them into a general power function.

Problem 5: How it all connects

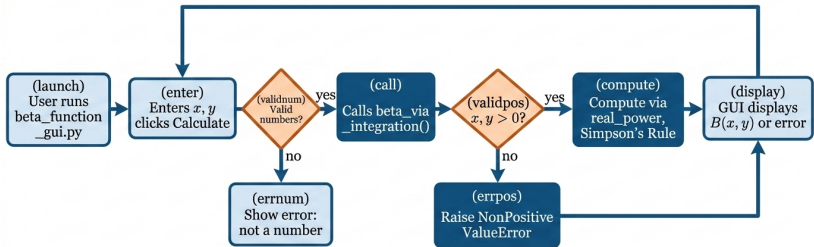


Figure: `beta_function_gui.py` handles only interaction; all validation logic and math live in `beta_function_scratch.py` – kept separate so the backend is independently testable.

Problem 5: Numerical Algorithm – Built From Scratch

Chain of subordinate functions (each built only from $+$, $-$, $*$, $/$, loops) in `beta_function_scratch.py`:

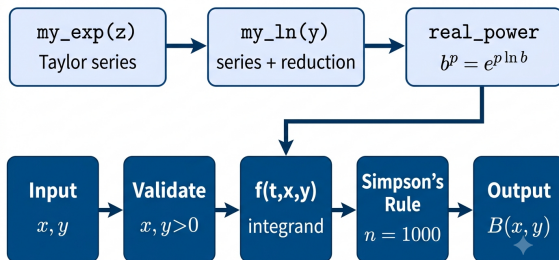


Figure: flow chart of implementation

- **my_exp(z)**: Taylor series for e^z , with *range reduction* – z is halved until small, series applied, result squared back up.

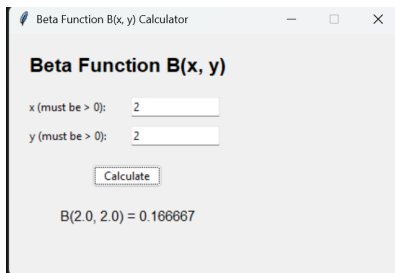
Promblem 5: Sub-ordinate Functions

- ▶ **my_ln(y)**: series $\ln(y) = 2(u + u^3/3 + u^5/5 + \dots)$, $u = \frac{y-1}{y+1}$, with range reduction toward $y \approx 1$.
- ▶ **real_power(b, p)**: $b^p = e^{p \cdot \ln(b)}$ – needed since x, y are real, not just integers.
- ▶ **Stopping criterion**: fixed 30 Taylor terms (`my_exp`), fixed 60 series terms (`my_ln`), and $n = 1000$ subintervals (Simpson's Rule) – chosen to exceed REQ-004's 6-digit precision target.

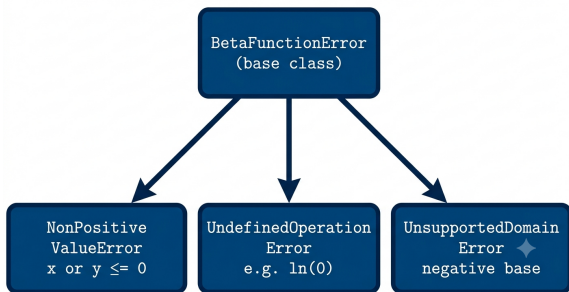
Problem 5: Tkinter GUI

Design:

- ▶ Thin wrapper – GUI `beta_function_gui.py` file has *no math logic*, only calls `beta_via_integration()` from the scratch module
- ▶ Entry fields for x , y ; Calculate button; Enter key bound as shortcut
- ▶ Result echoed alongside inputs (REQ-011)
- ▶ Small results auto-switch to scientific notation (e.g. 1.59×10^{-12}), avoiding false "0.000000"



Problem 5: Exception Handling



Why a hierarchy, not one generic exception: the GUI catches only `BetaFunctionError` once, covering all three subtypes, while each message stays specific to the actual problem (REQ-009). Positivity errors even name which variable failed: *"x must be strictly positive, but x = -1."*

Problem 5: Verification Cases

Each result compared against the true value via

$$B(x, y) = \Gamma(x)\Gamma(y)/\Gamma(x + y):$$

Case	Computed	True value	Error
$B(2, 3)$	0.08333333	0.08333333	0.0000%
$B(3, 2)$ symmetry	0.08333333	0.08333333	0.0000%
$B(10, 50)$ tiny	1.5916×10^{-12}	1.5916×10^{-12}	0.0000%
$B(5, 5)$	0.00158730	0.00158730	0.0000%
$B(0.5, 0.5)$	3.27189456	3.14159265	4.1476%

Discovered limitation: for $x, y \geq 1$, precision is excellent (error $< 0.01\%$). For $x, y < 1$, the integrand's edge singularity is under-resolved by Simpson's Rule, so REQ-004's 6-digit target is **not met** there – documented honestly, not hidden.

Problem 5: Verification Cases

Also verified live : input error, recovery on the next calculation and many more verification cases with easy to interpret for user.

The image displays two side-by-side screenshots of a web application titled "Beta Function B(x, y) Calculator". Each window contains the title "Beta Function B(x, y)" and two input fields: "x (must be > 0):" and "y (must be > 0):". Below the inputs is a "Calculate" button. The left window shows the input "1" in the y field and an error message: "Please enter a numeric value greater than 0 for x instead of \"." The right window shows both input fields empty and an error message: "Error: Please enter a valid number for x instead of \", and enter a valid number for y instead of \"."

fig :if x is empty

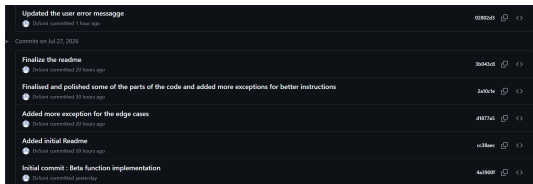
fig: if both the field is empty

Problem 6: Version Control

Public repository: <https://github.com/DvSoni/SOEN-6011-SOFTWARE-ENGINEERING-PROCESSES>

README includes:

- ▶ What the tool does, file structure
- ▶ How to run it
- ▶ How the calculation works
- ▶ Error handling, known limitations



Commit message examples:

- ▶ "Fix missing x-positivity check"
- ▶ "Refactor exceptions into a hierarchy"
- ▶ "Fix formatting for tiny results"

Each states *what* changed and *why*, not generic messages like "update" or "fix".

Problem 7: Requirements – Unchanged & Modified

ID	Status	Requirement
REQ-001, 002, 004, 006, 007, 011	Unchanged	Still verified true for the from-scratch implementation (accept x, y ; compute $B(x, y)$; ≥ 6 sig. digits*; < 1 sec; standard library only; echo inputs with result)
REQ-005	Modified	CLI text prompt $\rightarrow$ labeled GUI input fields for x and y
REQ-008	Modified	Now also excludes Python's <code>**</code> operator and <code>math</code> module, not just external libraries
REQ-009	Modified	Now requires a distinct custom exception type per category of invalid input, not just a generic message
REQ-010	Modified	CLI $\rightarrow$ GUI, still no IDE dependency to run

*Verified true for $x, y \geq 1$ (see Problem 5).

Known exception for $x, y < 1$ near the integral's singularity.

Problem 7: New Requirements

ID	Requirement (EARS format)
REQ-012	The system shall provide a graphical interface with labeled entry fields for x and y and a "Calculate" button.
REQ-013	If x is invalid and y is invalid at the same time, then the system shall report both problems in a single combined error message.
REQ-014	The system shall support pressing the Enter key as an alternative to clicking the Calculate button.
REQ-015	When the computed result's absolute value is smaller than 10^{-6} , the system shall display the result in scientific notation instead of fixed decimal notation.

Why these emerged: REQ-012–014 came from building the GUI itself; REQ-013 and REQ-015 were discovered through testing edge cases (both fields empty; very small results silently showing as "0.000000") – not anticipated in D1.

Problem 7: Updated Assumptions

- ▶ **A1–A3 (unchanged):** finite double-precision reals; one calculation at a time; standard Python 3 environment.
- ▶ **A4 (revised):** "Trustworthy computation" (REQ-008) now means avoiding Python's own built-in exponent/math functions too, not just external libraries – the persona's distrust of "black boxes" applies equally to hidden built-ins.
- ▶ **A5 (unchanged):** 6 significant digits remains the precision target.
- ▶ **A6 (new):** Negative bases are assumed unnecessary for this function's valid domain ($x, y > 0$), so `UnsupportedDomainError` is a safety net, not an expected user-facing case.
- ▶ **A7 (new):** For $x, y < 1$, precision below the 6-digit target is accepted as a known limitation of the fixed $n = 1000$ Simpson's Rule near the integral's edge singularity, pending a future refinement (e.g., adaptive step size).

GAI use – Problem 5 (CASTROFF)

Prompt type: Asking it to build code under strict constraints

Prompt I used (for the math functions):

*"I can't use ** or the math module for this part. Can you help me build my own e^z , $\ln(y)$, and a power function that works for any real exponent, using just basic loops and arithmetic? Also show me it actually matches Python's real answers."*

What I got back: working `my_exp`, `my_ln`, and `real_power` functions, plus a quick test comparing them to Python's built-ins (matched almost exactly). I asked a follow-up to make my error messages more specific instead of one generic error, which is where the `BetaFunctionError` hierarchy came from.

Small prompt (for the flowchart diagram):

"Can you turn this into a simple flow diagram – user opens the GUI, enters x and y , and show how it connects to the backend file that does the actual math and error checking?"

What I got back: a two-row diagram (GUI steps on top, backend steps below) with decision boxes for the validation checks.

GAI use – Problem 7 (CASTROFF)

Prompt type: Asking it to compare and update my old work

Prompt I used:

"Here are my D1 requirements (pasted REQ-001 to REQ-011). Now that I've built the GUI and the from-scratch math for D2, can you go through and tell me which requirements are still true, which ones need to be reworded, and if anything new is missing that I didn't think of in D1? Keep the old ID numbers the same, only add new numbers at the end."

What I got back: a list split into three groups – unchanged, modified, and brand new requirements. I didn't just copy this though – I went back and rewrote REQ-013 and REQ-015 myself to match the actual bugs I found while testing (like both fields being empty at once, and tiny results showing as 0.000000), instead of just keeping the tool's first draft wording.

Attributions

- ▶ ISO/IEC/IEEE 29148:2018, *Systems and software engineering – Life cycle processes – Requirements engineering*.
- ▶ Mavin, A., Wilkinson, P., Harwood, A., & Novak, M. (2009). Easy approach to requirements syntax (EARS). *17th IEEE International Requirements Engineering Conference*.
- ▶ Press, W. H., Teukolsky, S. A., Vetterling, W. T., & Flannery, B. P. (2007). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.). Cambridge University Press. (Taylor series range reduction; Simpson's Rule.)
- ▶ Abramowitz, M., & Stegun, I. A. (1964). *Handbook of Mathematical Functions*. National Bureau of Standards. (Beta function identities; series expansions for exp and ln.)
- ▶ Python Software Foundation. *tkinter — Python interface to Tcl/Tk*. Python 3 documentation.
- ▶ Python Software Foundation. *Built-in Exceptions*. Python 3 documentation. (Custom exception hierarchy design.)
- ▶ Beams, C. (2014). *How to Write a Git Commit Message*. cbea.ms.

Thank you

Questions?